HelpTask

Subject: Web Programming ASP

Subject professor: Luka Lukić

Student : Nikola Đurić 79/21

Contents

Introduction to project 3

Database 4

Controllers 5

UseCases 11

Test User Credentials 18

Introduction to project

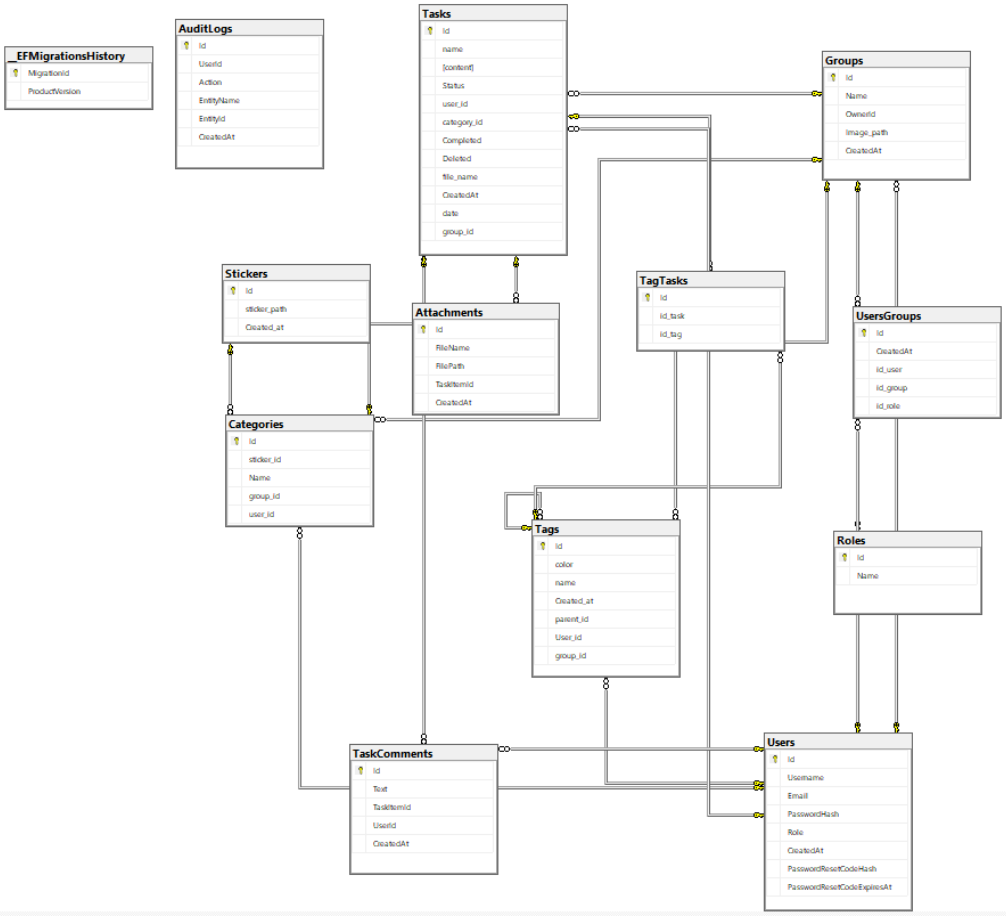
HelpTask is a modern task management application designed to help users organize and manage their tasks. Tasks can be categorized using Categories, Tags, or both.

The application provides two working modes: **Private Mode** and **Group Mode**. Private Mode is intended for managing personal tasks, while Group Mode enables collaborative task management. Both modes share the same user interface, but Group Mode introduces user roles: **Owner** and **Member**.

The group owner has permission to create, update, and delete categories and tags, as well as manage the group by changing its name, inviting new members, or removing existing ones. Group members can create and manage tasks within the categories and tags defined by the owner.

Users can attach files to tasks and download them when needed. Additionally, every user can create their own group and invite other users to collaborate on shared tasks.

Database



Picture above is the diagram of Database.

Controllers

1. Auth Controller

The **Auth Controller** is responsible for user authentication and account management. It allows users to register, log in, and reset their passwords. During login, a JWT token is generated and returned to the client for authenticated requests.

- **POST Register** (creates a new user account after validating the input and hashing the password)
- **POST Login** (authenticates the user and returns a JWT token)
- **POST Forgot Password** (generates a password reset code and sends it to the user's email address)
- **POST Reset Password** (verifies the reset code and updates the user's password)

2. Attachment Controller

The **Attachment Controller** is responsible for managing files attached to tasks. It allows authenticated users to upload files, download files, and retrieve all attachments for a specific task.

- **POST** (uploads a file for a specific task, validates file size, saves the file on the server, and stores attachment data in the database)
- **GET** (returns all attachments for a specific task)
- **GET Download** (downloads a selected attachment if it belongs to the requested task and user)

3. Categories Controller

The **Categories Controller** is responsible for managing task categories. It supports both private categories and group categories. In private mode, users can manage only their own categories. In group mode, all group members can view categories, while only the group owner can create, update, or delete them.

- **GET** (returns all categories for the authenticated user or for a selected group)
- **GET By Id** (returns a specific category by its id)
- **POST** (creates a new category; in group mode, only the group owner can perform this action)
- **PUT** (updates an existing category; in group mode, only the group owner can perform this action)
- **DELETE** (deletes an existing category; in group mode, only the group owner can perform this action)

5. Group Controller

The **Group Controller** is responsible for managing user groups. It allows users to create groups, retrieve the groups they belong to, view information about a specific group, and delete groups they own.

- **GET** (returns all groups the authenticated user is a member of, including the number of group members)
- **GET By Id** (returns information about a specific group if the authenticated user is a member)

- **POST** (creates a new group and automatically assigns the creator as the group owner)
- **DELETE** (deletes a group; only the group owner is allowed to perform this action)

6. Seeder Controller

The **Seeder Controller** is responsible for populating the database with sample data for testing and development purposes. It creates predefined roles, users, groups, categories, tags, stickers, tasks, and their relationships. It also provides an option to reset the database to its initial state.

- **POST** (populates the database with sample data, including users, groups, categories, tags, tasks, and related entities)
- **POST Reset** (deletes the current database, recreates it using the latest migrations, and restores the initial database structure)

7. Stickers Controller

The **Stickers Controller** is responsible for retrieving sticker resources used by task categories. It allows authenticated users to retrieve all available stickers or a specific sticker by its identifier.

- **GET** (returns a list of all available stickers)
- **GET By Id** (returns a specific sticker by its id)

8. Tags Controller

The **Tags Controller** is responsible for managing task tags. It supports both private tags and group tags. In private mode, users can manage

only their own tags. In group mode, all group members can view tags, while only the group owner can create, update, or delete them.

- **GET** (returns all private tags for a user or all tags for a selected group)
- **GET By Id** (returns a specific tag by its id)
- **POST** (creates a new tag; in group mode, only the group owner can perform this action)
- **PUT** (updates an existing tag; in group mode, only the group owner can perform this action)
- **DELETE** (deletes an existing tag; in group mode, only the group owner can perform this action)

9. TagTasks Controller

The **TagTasks Controller** is responsible for managing the relationship between tasks and tags. It allows authenticated users to assign tags to tasks, remove assigned tags, and retrieve all task-tag relationships associated with their tags.

- **GET** (returns all task-tag relationships for the authenticated user)
- **POST** (creates a new relationship between a task and a tag after validating ownership)
- **DELETE** (removes the relationship between a specific task and tag)

10. Tasks Controller

The **Tasks Controller** is responsible for managing tasks in the application. It allows authenticated users to create, retrieve, update,

and delete tasks. It supports both private tasks and group tasks, with filtering by keyword, status, group, and tag. The controller also supports pagination and includes related data such as category, sticker, tags, attachments, comments, and assigned user information.

- **POST** (creates a new task for the authenticated user and stores an audit log)
- **GET** (returns a paginated list of tasks with optional filtering by keyword, status, group, and tag)
- **GET By Id** (returns a specific task with its related comments and attachments)
- **PUT** (updates an existing task and stores an audit log)
- **DELETE** (deletes an existing task and stores an audit log)

11. Users Controller

The **Users Controller** is responsible for retrieving user information. It allows clients to retrieve all registered users or search for users by their username or email address.

- **GET** (returns a list of all registered users)
- **GET Search** (returns users whose username or email matches the provided search term)

12. UsersGroup Controller

The **UsersGroup Controller** is responsible for managing the relationship between users and groups. It allows retrieving group memberships, listing users inside a specific group, adding users to a group, and

removing users from a group. Only the group owner is allowed to add or remove group members.

- **GET** (returns all user-group relationships)
- **GET By Group Id** (returns all users that belong to a specific group)
- **POST** (adds one or more users to a group; only the group owner can perform this action)
- **DELETE** (removes one or more users from a group; only the group owner can perform this action)

UseCases

IV. Use Cases

IV. Use Cases

The application currently implements **41 use cases** that are available through its REST API. Access to these use cases is controlled using JWT authentication and authorization. Users are allowed to perform operations only on resources they own, while group-related operations are additionally restricted according to the user's role within the group (Owner or Member). Group owners are allowed to manage groups, categories, tags, and group members, while group members can manage tasks within the group.

The list below contains all implemented use cases, their IDs, names, and descriptions.

1. Register User Command

ID: 1
Role: Registers a new user account.

2. Login User Command

ID: 2
Role: Authenticates a user and returns a JWT token.

3. Forgot Password Command

ID: 3
Role: Generates and sends a password reset code to the user's email address.

4. Reset Password Command

ID: 4
Role: Verifies the reset code and updates the user's password.

5. Create Task Command

ID: 5
Role: Creates a new task.

6. Update Task Command

ID: 6
Role: Updates an existing task.

7. Delete Task Command

ID: 7
Role: Deletes an existing task.

8. Get Tasks Query

ID: 8
Role: Retrieves tasks with support for filtering, searching, and pagination.

9. Get Task By Id Query

ID: 9
Role: Retrieves a specific task together with its related information.

10. Create Category Command

ID: 10
Role: Creates a new category.

11. Update Category Command

ID: 11
Role: Updates an existing category.

12. Delete Category Command

ID: 12
Role: Deletes an existing category.

13. Get Categories Query

ID: 13
Role: Retrieves all categories for the authenticated user or selected group.

14. Get Category By Id Query

ID: 14
Role: Retrieves a specific category.

15. Create Tag Command

ID: 15
Role: Creates a new tag.

16. Update Tag Command

ID: 16
Role: Updates an existing tag.

17. Delete Tag Command

ID: 17
Role: Deletes an existing tag.

18. Get Tags Query

ID: 18
Role: Retrieves all tags for the authenticated user or selected group.

19. Get Tag By Id Query

ID: 19
Role: Retrieves a specific tag.

20. Assign Tag To Task Command

ID: 20
Role: Assigns a tag to a task.

21. Remove Tag From Task Command

ID: 21
Role: Removes a tag from a task.

22. Get Task Tags Query

ID: 22
Role: Retrieves all task-tag relationships.

23. Create Comment Command

ID: 23
Role: Adds a comment to a task.

24. Get Task Comments Query

ID: 24
Role: Retrieves all comments for a task.

25. Upload Attachment Command

ID: 25
Role: Uploads a file attachment to a task.

26. Get Task Attachments Query

ID: 26
Role: Retrieves all attachments for a task.

27. Download Attachment Query

ID: 27
Role: Downloads a selected task attachment.

28. Create Group Command

ID: 28
Role: Creates a new group.

29. Delete Group Command

ID: 29
Role: Deletes a group owned by the authenticated user.

30. Get User Groups Query

ID: 30
Role: Retrieves all groups the authenticated user belongs to.

31. Get Group By Id Query

ID: 31
Role: Retrieves information about a specific group.

32. Get Group Memberships Query

ID: 32
Role: Retrieves all user-group relationships.

33. Get Group Members Query

ID: 33
Role: Retrieves all members of a specific group.

34. Add Users To Group Command

ID: 34
Role: Adds one or more users to a group. Only the group owner is allowed to perform this action.

35. Remove Users From Group Command

ID: 35
Role: Removes one or more users from a group. Only the group owner is allowed to perform this action.

36. Get Users Query

ID: 36
Role: Retrieves all registered users.

37. Search Users Query

ID: 37
Role: Searches users by username or email address.

38. Get Stickers Query

ID: 38
Role: Retrieves all available stickers.

39. Get Sticker By Id Query

ID: 39
Role: Retrieves a specific sticker.

40. Seed Database Command

ID: 40
Role: Populates the database with sample data.

41. Reset Database Command

ID: 41
Role: Resets the database and recreates the initial sample data.

V. Test User Credentials

The application database is populated with sample users through the Seeder endpoint. The following credentials can be used for testing:

Username	E-mail	Password	Notes
nikola	nikola@test.com	123456	Owner of the **TaskFlow Team** group
marko	marko@test.com	123456	Member of **TaskFlow Team**, Owner of **Frontend Developers**
ana	ana@test.com	123456	Member of **Frontend Developers**, Owner of **Backend Developers**
jelena	jelena@test.com	123456	Member of **Backend Developers**, Owner of **QA Team**
milos	milos@test.com	123456	Member of **QA Team**, Owner of **Marketing**

All remaining users created by the Seeder endpoint use the password **123456**.

The application does not implement predefined user use cases. Instead, access to API endpoints is controlled through JWT authentication, while group-related operations are authorized according to the user's role within the group (Owner or Member). Only group owners are allowed to manage categories, tags, and group members.